

# 语音/视频通话模块 (Voice) 详细报告

## 一、需求与设计

### 1.1 功能需求

| 需求编号     | 需求描述                            | 优先级 |
|----------|---------------------------------|-----|
| VOICE-01 | 一对一语音通话 (WebRTC Audio)          | P0  |
| VOICE-02 | 一对一视频通话 (WebRTC Audio+Video)    | P1  |
| VOICE-03 | 通话状态管理 (RINGING→ACCEPTED→ENDED) | P0  |
| VOICE-04 | WebSocket 信令传输 (SDP + ICE)      | P0  |
| VOICE-05 | ICE 服务器配置 (STUN/TURN)           | P1  |
| VOICE-06 | 忙线检测 (同一用户只能在一个通话中)             | P1  |
| VOICE-07 | 过期通话自动清理                        | P1  |
| VOICE-08 | 通话状态轮询备份 (WebSocket断开时)         | P1  |

### 1.2 设计决策

- W3C Perfect Negotiation 模式** — 解决 WebRTC 双向呼叫时的 "glare" 竞技问题
- 主叫方延迟添加音轨** — 主叫方在收到被叫方 offer 后才 addTrack，确保双方 SDP 都含音频
- WebSocket + REST 双重通道** — 信令实时走 WebSocket，状态轮询走 REST (3秒间隔备份)
- 数据库驱动的忙线检测** — 替代旧的 `BusyUserRegistry` 内存方案，解决 WebSocket 断开后状态残留
- 双向清理** — `start()` 时清理双方的旧活跃通话

## 二、后端设计

### 2.1 核心文件清单

| 文件                         | 行数   | 位置                                                              |
|----------------------------|------|-----------------------------------------------------------------|
| VoiceResource.java         | 63   | src/main/java/com/example/chat/voice/VoiceResource.java         |
| VoiceService.java          | 113  | src/main/java/com/example/chat/voice/VoiceService.java          |
| VoiceDao.java              | ~120 | src/main/java/com/example/chat/voice/VoiceDao.java              |
| VoiceCallNotifier.java     | 51   | src/main/java/com/example/chat/voice/VoiceCallNotifier.java     |
| IceServerConfig.java       | ~20  | src/main/java/com/example/chat/voice/IceServerConfig.java       |
| VoiceSignal.java           | ~15  | src/main/java/com/example/chat/voice/VoiceSignal.java           |
| VoiceCallStartRequest.java | ~10  | src/main/java/com/example/chat/voice/VoiceCallStartRequest.java |

### 2.2 REST API 端点

| HTTP | 路径                               | 功能         |
|------|----------------------------------|------------|
| POST | /api/voice/calls/{calleeId}      | 发起通话       |
| GET  | /api/voice/calls/incoming        | 查询来电       |
| GET  | /api/voice/calls/{callId}        | 查询通话状态     |
| POST | /api/voice/calls/{callId}/accept | 接听         |
| POST | /api/voice/calls/{callId}/reject | 拒绝         |
| POST | /api/voice/calls/{callId}/end    | 挂断         |
| GET  | /api/voice/ice-servers           | 获取ICE服务器配置 |

代码位置: `src/main/java/com/example/chat/voice/VoiceResource.java`

## 2.3 VoiceService — 核心业务逻辑

发起通话 (`start()`):

1. 验证 `callerId != calleeId` (不能呼叫自己)
2. `cleanupStaleActiveCalls()` – 清理过期通话 (`RINGING>2min`, `ACCEPTED>30min`)
3. `clearActiveCallsForUser(callerId)` – 清理主叫旧通话
4. `clearActiveCallsForUser(calleeId)` – 清理被叫旧通话 ← 关键修复
5. `isUserInCall(callerId)` → 忙线检查
6. `isUserInCall(calleeId)` → 对方忙线检查
7. `createCall()` → 创建新通话 (`status=RINGING`)
8. `notifyCallInvite()` → `WebSocket` 推送来电通知

代码位置: `src/main/java/com/example/chat/voice/VoiceService.java` 第19-42行

通话状态流转:

```
RINGING → (接听) → ACCEPTED → (挂断) → ENDED
RINGING → (拒绝) → REJECTED
RINGING → (超时2分钟) → MISSED (由cleanupStaleActiveCalls处理)
```

## 2.4 VoiceCallNotifier — WebSocket 推送

通过 `SocketRegistry` 向 channel "`voice`" 发送信令通知:

- `notifyCallInvite()` — 来电邀请 (type: "call-invite")
- `notifyCallAccepted()` — 对方已接听 (type: "call-accepted")
- `notifyCallRejected()` — 对方已拒绝 (type: "call-rejected")
- `notifyCallEnded()` — 通话已结束 (type: "call-ended")

代码位置: `src/main/java/com/example/chat/voice/VoiceCallNotifier.java`

## 2.5 ICE 服务器配置

支持 STUN 和 TURN 服务器配置, 通过 `app.properties` 或环境变量:

- `rtc.stunUrls` — 默认 `stun:stun.l.google.com:19302`
- `rtc.turnUrl` — TURN服务器地址
- `rtc.turnUsername` — TURN用户名
- `rtc.turnCredential` — TURN凭证

代码位置: `src/main/java/com/example/chat/voice/IceServerConfig.java` REST端点: GET `/api/voice/ice-servers` → 前端 `loadIceServers()` 前端调用: `src/main/webapp/assets/js/voice.js` 第 37-40行

## 三、数据库设计

### 3.1 voice\_call\_sessions 表

| 列名               | 类型          | 约束                        | 说明                                     |
|------------------|-------------|---------------------------|----------------------------------------|
| id               | BIGINT      | PK, AUTO_INCREMENT        | 通话ID                                   |
| caller_id        | BIGINT      | FK → users, NOT NULL      | 主叫方                                    |
| callee_id        | BIGINT      | FK → users, NOT NULL      | 被叫方                                    |
| call_mode        | VARCHAR(20) | NOT NULL, DEFAULT 'audio' | audio/video                            |
| status           | VARCHAR(20) | NOT NULL                  | RINGING/ACCEPTED/REJECTED/ENDED/MISSED |
| started_at       | TIMESTAMP   | NOT NULL                  | 开始时间                                   |
| accepted_at      | TIMESTAMP   | NULL                      | 接听时间                                   |
| ended_at         | TIMESTAMP   | NULL                      | 结束时间                                   |
| duration_seconds | INT         | DEFAULT 0                 | 通话时长                                   |

建表SQL: `src/main/resources/schema.sql` 第227-239行 `call_mode` 列由 `SchemaMigrator` 自动添加:  
`src/main/java/com/example/chat/config/SchemaMigrator.java` — `ensureVoiceCallModeColumn()`

### 3.2 VoiceDao 核心SQL

- `createCall()` — INSERT voice\_call\_sessions (status=RINGING)
- `isUserInCall()` — SELECT 1 WHERE (caller\_id=? OR callee\_id=?) AND status IN ('RINGING','ACCEPTED')
- `clearActiveCallsForUser()` — UPDATE status='MISSED' WHERE status IN ('RINGING','ACCEPTED')
- `cleanupStaleActiveCalls()` — RINGING→MISSED if >2min, ACCEPTED→ENDED if >30min
- `findIncomingRinging()` — SELECT 最新 RINGING 通话 for a callee

代码位置: `src/main/java/com/example/chat/voice/VoiceDao.java`

## 四、前端设计 — WebRTC 完美协商

### 4.1 文件结构

| 文件       | 行数  | 位置                                              | 职责              |
|----------|-----|-------------------------------------------------|-----------------|
| voice.js | 273 | <code>src/main/webapp/assets/js/voice.js</code> | 主入口: 通话流程+W3C协商 |



```

    // 视频模式：显示远程视频
    if (activeCallMode === "video") {
        $("#remoteVideoStream").srcObject = e.streams[0];
        $("#videoCallStage").hidden = false;
    }
};

peer.onnegotiationneeded = async () => {
    makingOffer = true;
    await peer.setLocalDescription();
    sendSignal(activePeerId, JSON.stringify(peer.localDescription));
    makingOffer = false;
};

peer.onicecandidate = e => {
    if (e.candidate) sendSignal(activePeerId, JSON.stringify({candidate:
e.candidate}));
};
}

```

代码位置：src/main/webapp/assets/js/voice.js 第64-102行

### handleRemoteSdp() — SDP 协商核心

```

async function handleRemoteSdp(desc, fromUserId) {
    const isOffer = desc.type === "offer";
    // 竞争检测 (glare detection)
    const offerCollision = isOffer && (makingOffer || pc.signalingState !== "stable");
    ignoreOffer = !isOffer && pc.signalingState === "stable";
    if (offerCollision || ignoreOffer) return;

    // ★ 关键：主叫方收到被叫offer后才addTrack
    if (isOffer && localStream && pc.getSenders().length === 0) {
        localStream.getTracks().forEach(t => pc.addTrack(t, localStream));
    }
    await pc.setRemoteDescription(new RTCSessionDescription(desc));
    if (isOffer) {
        await pc.setLocalDescription(); // 生成answer
        sendSignal(fromUserId, JSON.stringify(pc.localDescription));
    }
}

```

代码位置：src/main/webapp/assets/js/voice.js 第154-172行

### startPrivateCall(mode) — 发起通话

- 获取本地媒体流 (audio 或 audio+video)
- 创建空 RTCPeerConnection (不加 track)
- 通过 REST 创建通话
- 通过 WebSocket 发送 call-invite 代码位置：src/main/webapp/assets/js/voice.js 第176-193行

### acceptVoiceCall() — 接听

- 获取本地媒体流
- 创建 `RTCPeerConnection` 并 **立即** `addTrack`
- `onnegotiationneeded` 自动触发 → `createOffer` → 发送给主叫 代码位置：  
`src/main/webapp/assets/js/voice.js` 第196-212行

## 4.4 视频通话

### HTML 元素：

```
<div id="videoCallStage" class="video-call-stage" hidden>
  <video id="remoteVideoStream" class="remote-video" autoplay playsinline></video>
  <video id="localVideoStream" class="local-video" autoplay playsinline muted>
</video>
</div>
```

代码位置：`src/main/webapp/app.html` — `#voiceDialog` 内

### 视频模式逻辑：

- `getLocalStream("video")` — `getUserMedia({audio:true, video:{width:640,height:480}})`
- 设置 `localVideoStream.srcObject = localStream`
- `ontrack` 中检测 `activeCallMode === "video"` → 设置 `remoteVideoStream.srcObject`

代码位置：`src/main/webapp/assets/js/voice.js` 第42-56行、第77-81行

## 4.5 轮询备份

```
// 每3秒轮询通话状态 (WebSocket断开时的备份)
setInterval(async () => {
  if (activeCallId && activePeerId) {
    const call = await ChatApi.get(`/voice/calls/${activeCallId}`);
    if (["ENDED", "REJECTED", "MISSED"].includes(call.status)) cleanupCall(true);
  }
  if (!activeCallId && !pendingIncomingCall) {
    const call = await ChatApi.get("/voice/calls/incoming");
    if (call?.status === "RINGING") {
      // 显示来电弹窗
    }
  }
}, 3000);
```

代码位置：`src/main/webapp/assets/js/voice.js` 第243-258行

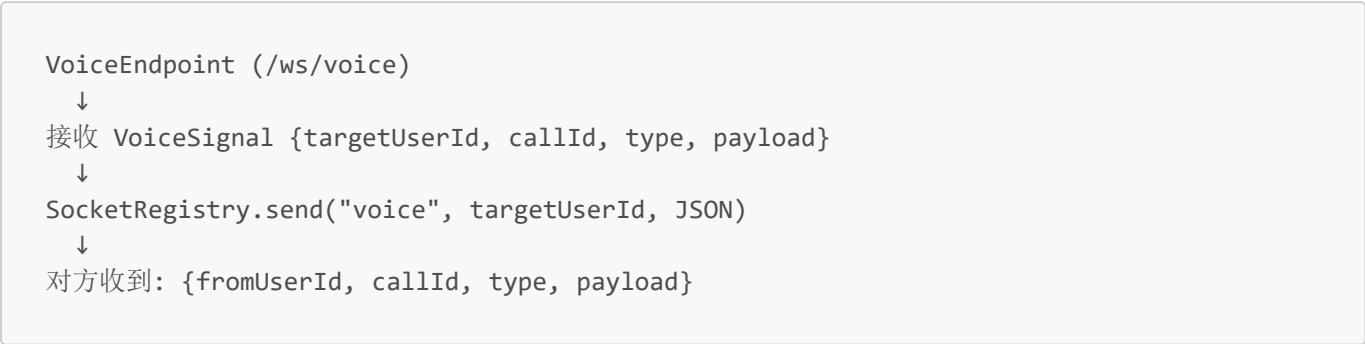
## 4.6 页面关闭处理

```
// beforeunload: 使用 sendBeacon 发送挂断请求
window.addEventListener("beforeunload", () => {
  if (activeCallId) navigator.sendBeacon(
    "api/voice/calls/" + activeCallId + "/end",
    new Blob(["{}"], {type:"application/json"})
  );
});
```

```
    );
  });
```

代码位置：[src/main/webapp/assets/js/voice.js](#) 第268-270行

## 五、WebSocket 信令中继



- SDP (offer/answer): 完整 SDP 字符串通过 JSON 传输
- ICE candidate: `{candidate: ...}` 对象
- 类型路由: "call-invite" / "call-accepted" / "call-rejected" / "call-ended"

代码位置：

- 后端：[src/main/java/com/example/chat/websocket/VoiceEndpoint.java](#) (47行)
- 前端发送：[src/main/webapp/assets/js/voice.js](#) — `sendSignal()` (115-118行)
- 前端接收：[src/main/webapp/assets/js/voice.js](#) — `onSignal()` (120-149行)

## 六、已知问题与解决方案

| 问题                | 根因                                     | 解决方案                       | 代码位置                                                                    |
|-------------------|----------------------------------------|----------------------------|-------------------------------------------------------------------------|
| 单向音频<br>(只有一方能听到) | 主叫方提前 addTrack 触发 negotiate，被叫收到空offer | 主叫方延迟到收到被叫 offer后才addTrack | <a href="#">voice.js</a> 第163-165行                                      |
| "对方忙线"误报          | 内存 BusyUserRegistry WebSocket断开后不清除    | 改用DB查询 isUserInCall()      | <a href="#">VoiceService.java</a> <code>start()</code>                  |
| 通话状态残留            | clearActiveCallsForUser 只清理主叫          | 启动时清理双方                    | <a href="#">VoiceService.java</a> 第26-27行                               |
| 过期通话不清理           | RINGING清理窗口过长(1分钟)                     | 缩短为2分钟，并同时清理双方             | <a href="#">VoiceDao.java</a><br><code>cleanupStaleActiveCalls()</code> |
| device in use     | 同一浏览器两个标签页                             | 使用不同浏览器测试 (Chrome+Edge)    | —                                                                       |